Project Report

Password Analyzer

Name: Shivang Sharma

Registration no: 25BCE10160

Faculty: Badla Pawan Babu

Slot: C14+E11+E12

1. Introduction

This project details the development of a cybersecurity tool “Password Analyzer”, built using python which is a part of flipped course evaluation requiring an original technical solution. The primary goal of this project is to evaluate the password strength and give feedback to the user to strengthen their password.

2. Problem Statement

Weak passwords make general web users vulnerable to brute force and credential stuffing attacks. Users tend to rely on predictable and sequential patterns (e.g., “1234” or “aaa”) that might pass basic length checks but offer zero real life protection. This project is designed as a tool that provides feedback based on entropy rather than just character count.

3. Functional Requirements (FRs)

The system includes three major functional modules:

1. Input Processing: Capturing and sanitizing user passwords.
2. Risk Analysis Engine: Evaluating character diversity and patterns.
3. Dynamic Reporting: Displaying scores and security ratings.
4. Weighted Scoring Engine: Assigns values to uppercase, lowercase, digits, and special characters.
5. Pattern Recognition: Detects and penalizes sequential characters and character repetition.

4. Non-functional Requirements (NFRs)

Usability: Simple CLI for ease of interaction.
Reliability: Handling empty inputs without crashing.
Maintainability: Modular code for easy weightage updates.
Error Handling: Strategy to manage invalid string inputs.

5. System Architecture

The project follows a modular architectural design.

- UI Layer: main.py handles user interaction.
- Logic Layer: analyzer.py contains the PasswordAnalyzer class.
- Data Layer: Configuration dictionary for scoring weights.

6. Design Diagrams

6.1. Use Case Diagram

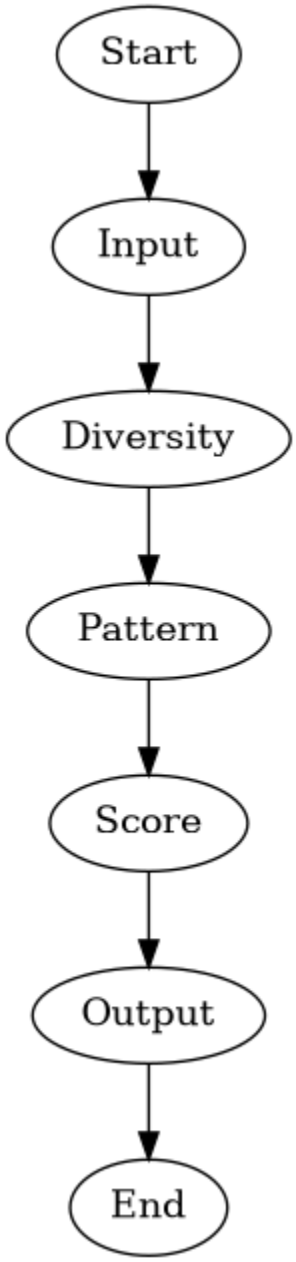
Purpose: To show the interactions between the user and the system functions.

- **Actor:** The End User.
- **System Boundary:** The Python CLI Application.
- **Use Cases:**
    - **Input Password:** The primary entry point for data.
    - **View Risk Analysis:** The internal process where the heuristic engine evaluates the string.
    - **Receive Security Feedback:** The output phase where actionable tips are displayed to the user.
    - **Exit System:** The formal termination of the program loop.

6.2. Workflow Diagram (Process Flow)

Purpose: To show the logical "path" the data takes from start to finish
Step 1 (Start): The user launches main.py.
Step 2 (Input): The system prompts for a password string.
Step 3 (Processing - Diversity Check): The PasswordAnalyzer scans for uppercase, lowercase, numbers, and symbols.
Step 4 (Processing - Pattern Check): The engine runs sub-routines to detect sequential strings (e.g., "123") or repeated characters (e.g., "aaa").

Step 5 (Decision): The system aggregates the score and identifies missing security elements.
Step 6 (Output): The CLI displays the final Risk Score, Star Rating, and a list of specific tips.



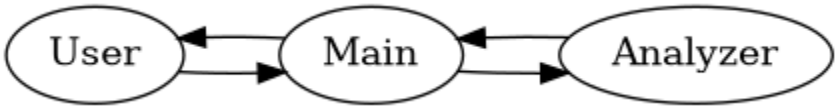
6.3. Sequence Diagram

Purpose: To show the step-by-step interaction between objects over time.
Participants:

- User: The person providing the input.
- Main Script (main.py): The controller handling the loop and display.
- Analyzer (analyzer.py): The core engine performing the calculations.

The Flow:

1. User sends input to Main Script.
2. Main Script calls the analyze method on the Analyzer object.
3. Analyzer internally calls _check_sequence() and _check_repetition() to calculate penalties.
4. Analyzer returns the score and feedback[] list to the Main Script.
5. Main Script calls get_rating(score) to determine the star level.
6. Main Script displays the final security report to the User.



6.4. Class/Component Diagram

Purpose: To show the technical structure and Object-Oriented Design

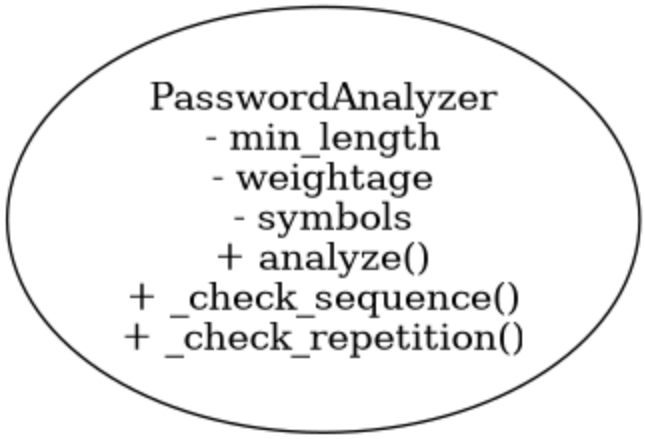
Class Name: PasswordAnalyzer.

Attributes (Data):

- min_length: Integer (Default: 10).
- weightage: Dictionary (Stores scores for different character types).
- symbols: String (Standard punctuation set).

Methods (Functions):

- analyze(password): The public interface that returns the total score and feedback list.
- _check_sequence(password): Private helper to identify keyboard patterns (Penalty logic).
- _check_repetition(password): Private helper to identify character spamming.



6.5. ER Diagram (Entity-Relationship Diagram)

Purpose: To show the data entities handled by the program

Entities & Attributes:

- User Session:
  - Session_ID (Primary Key)
  - Timestamp
- Password Entry:
  - Raw_Text (The input string)
  - Length
  - Has_Special_Chars (Boolean)
- Analysis Result:
  - Final_Score (Integer)
  - Risk_Level (String: Strong/Moderate/Weak)
  - Feedback_Tips (List of Strings)

Relationships:

- A User Session *creates* one or more Password Entries (1:N).
- Each Password Entry *generates* exactly one Analysis Result (1:1).

7. Design Decisions & Rationale

- Python Language: Chosen for its robust string library and readability.
- Weighted Scoring: Selected to ensure special characters carry more "weight" than simple letters.

8. Implementation Details

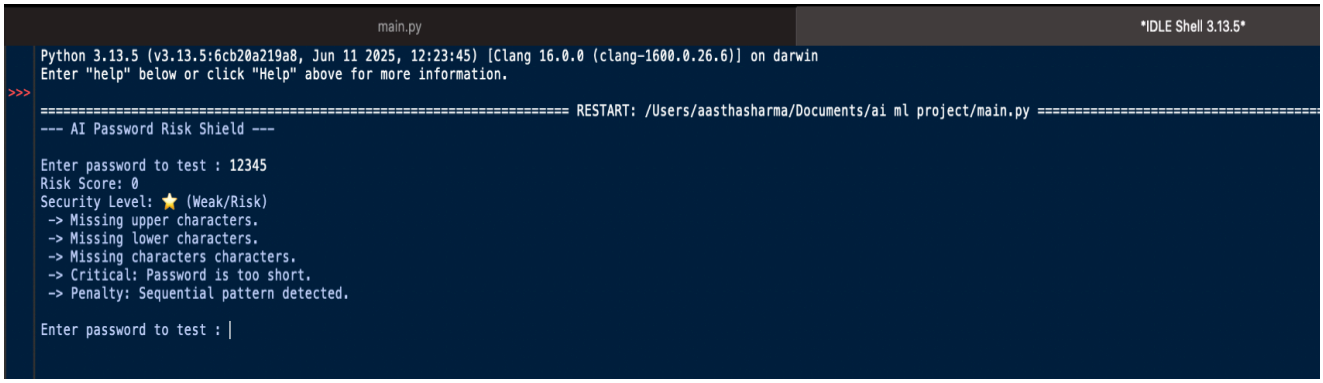
The project is implemented across two main Python files:

- analyzer.py: Contains the PasswordAnalyzer class which uses the string library to categorize characters.
- main.py: Handles the user loop and maps numerical scores to star ratings.

9. Screenshots / Results

1. The "Critical Risk" Case (Multiple Penalties)

- **Input:** 12345
- **What it proves:** This shows the system detects short length (< 6 characters) and sequential patterns.
- **Expected Output:** * Risk Score: (Negative or very low)
    - Security Level: ⭐ (Weak/Risk)
    - Feedback: "Critical: Password is too short", "Penalty: Sequential pattern detected", "Missing upper characters", etc.



2. The "Predictable" Case (Repetition Penalty)

- **Input:** AAAAAaaaaa111
- **What it proves:** Even though this is 13 characters long, it shows your repetition penalty logic.
- **Expected Output:**
    - Security Level: ⭐ (Weak/Risk)
    - Feedback: "Penalty: Repeating characters detected."

```
main.py                                                    *IDLE Shell 3.13.5*
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
========================================================================== RESTART: /Users/aasthasharma/Documents/ai ml project/main.py ==========================================================================
--- AI Password Risk Shield ---

Enter password to test : AAAAAaaaaa111
Risk Score: 55
Security Level: ⭐⭐⭐ (Moderate)
 -> Missing characters characters.
 -> Penalty: Repeating characters detected.

Enter password to test : |
```

3. The "Mixed Moderate" Case (Diversity but Patterned)

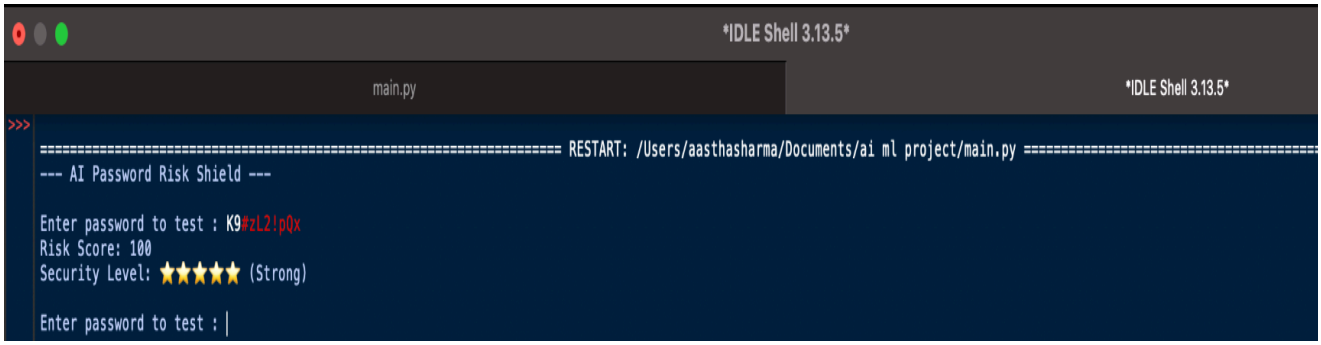
- **Input:** P@ssword123
- **What it proves:** This shows a user who tried to include symbols and numbers but used a common sequence("123").
- **Expected Output:**
  - Security Level: ⭐⭐⭐ (Moderate)
  - Feedback: "Penalty: Sequential pattern detected."

```
X                                  main.py                                                    *IDLE Shell 3.13.5*
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
========================================================================== RESTART: /Users/aasthasharma/Documents/ai ml project/main.py ==========================================
--- AI Password Risk Shield ---

Enter password to test : P@ssword123
Risk Score: 80
Security Level: ⭐⭐⭐⭐⭐ (Strong)
 -> Penalty: Sequential pattern detected.

Enter password to test : |
```

4. The "Ideal/Strong" Case (Maximum Score)

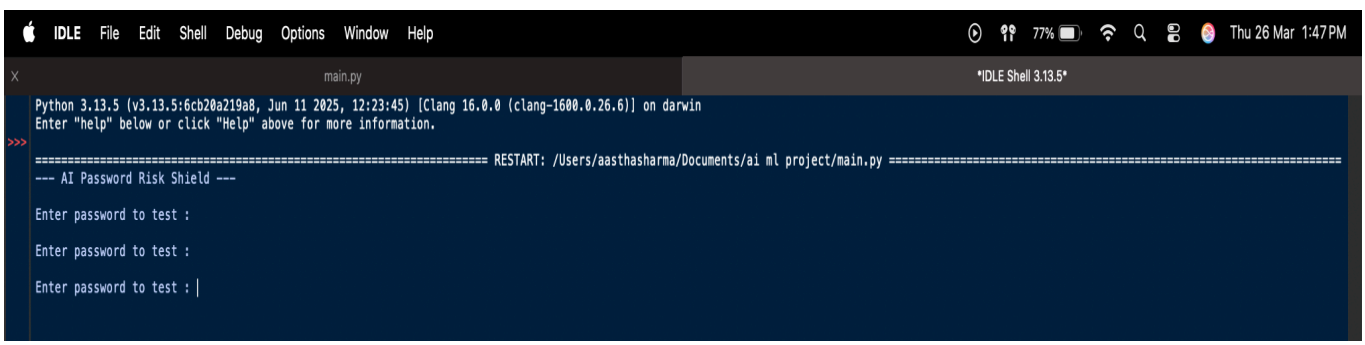
- **Input:** K9#zL2!pQx
- **What it proves:** This validates that a high-entropy password with no patterns receives the Length Bonus and Randomness Bonus.
- **Expected Output:**
  - Risk Score: 85+
  - Security Level: ⭐⭐⭐⭐⭐ (Strong)

- Feedback: (No negative tips should appear).



5. The "Edge Case" (Empty/Exit)

- **Input:** Just press Enter or type exit
- **What it proves:** Demonstrates robust error handling and a clean exit strategy, which are key non-functional requirements.



10. Testing Approach

Validation tests were performed using specific cases:

- **Sequential Test**: Inputting "abc123" to verify penalty application.
- **Complexity Test**: Inputting "K9#zL2!pQx" to verify high security rating.

11. Challenges Faced

- **Sequence Detection:** Implementing a check for both ascending ("abc") and descending ("cba") sequences required careful indexing logic.
- **Weight Balancing:** Ensuring that the "Randomness Bonus" didn't overshadow "Missing Character" penalties.

- **Entropy detection**: threshold for "randomness" without using complex external libraries, which was solved by calculating the unique character ratio against total length.

12. Learnings & Key Takeaways

- Applied course concepts in a real-world context, specifically learning how to implement modular logic and version control using Git
- Gained experience in Modular Programming and Object-Oriented Design in Python.
- Understood how heuristic scoring is more effective for security than simple Boolean checks.

13. Future Enhancements

- Integration with a Graphic User Interface (GUI).
- Adding a "Leaked Password" check using an API.
- Integration with a Commonly Breached Passwords API (like HaveIBeenPwned).

14. References

- VITyarthi Project Guidelines.
- Python Documentation (string module).